

Завдання на папері до вступних лекцій

Комп'ютерна схемотехніка та архітектура комп'ютерів · Лекція 1 «Вступ до комп'ютерної схемотехніки та її роль в ІТ»

Усе виконується ручкою або олівцем у зошиті. Калькулятор не потрібен — усі числа підібрані так, щоб рахувати в умі або стовпчиком. Блоки незалежні: їх можна давати в будь-якому порядку й у будь-якій кількості.

Блок А. Системи числення · ≈ 12 хв

- A1** Переведіть у двійкову систему методом послідовного ділення на 2. Обов'язково запишіть стовпчик остач — саме він оцінюється, а не лише відповідь: 19, 64, 100, 255.

.....

.....

.....

.....

.....

.....

- A2** Переведіть у десяткову систему, розписавши ваги розрядів: 1011_2 , 110110_2 , 1000000_2 , 1111111_2 .

.....

.....

.....

.....

.....

.....

- A3** Переведіть без жодних обчислень, лише розбиттям на четвірки: $11010110_2 \rightarrow$ шістнадцяткове, $01011010_2 \rightarrow$ шістнадцяткове, $0x2F \rightarrow$ двійкове, $0xC0 \rightarrow$ двійкове.

.....

.....

.....

.....

.....

.....

- A4** Додайте стовпчиком прямо у двійковій системі, а потім перевірте себе, перевівши доданки й результат у десяткову: $1011 + 0110$, $1111 + 0001$, $1100 + 1010$.

.....

.....

.....

.....

.....

.....

A5 Скільки щонайменше бітів потрібно, щоб закодувати:

.....

.....

.....

- а) 33 літери української абетки
- б) 7 днів тижня
- в) 365 днів року
- г) 1000 студентів коледжу

A6 Колір #FF8800 — запишіть його три складові десятковими числами. І навпаки: запишіть колір rgb(0, 128, 255) у шістнадцятковому вигляді.

.....

.....

.....

| Блок Б. Логіка й таблиці істинності · ≈ 12 хв

B1 Заповніть таблицю істинності для чотирьох елементів:

A	B	AND	OR	XOR	NAND
0	0				
0	1				
1	0				
1	1				

B2 Обчисліть значення виразу (A AND B) OR (NOT A) для всіх чотирьох комбінацій входів. Порівняйте стовпчик результату з базовими операціями з B1 — на що він схожий?

.....

.....

.....

B3 Скільки рядків матиме таблиця істинності для елемента з 3 входами? З 4 входами? З 8 входами? Запишіть загальну формулу.

.....

.....

.....

Б4 Схеми голосування: лампа має горіти, якщо ввімкнено хоча б два з трьох перемикачів А, В, С. Заповніть повну таблицю на вісім рядків.

А	В	С	Лампа
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Б5 Порівняйте стовпчик XOR із таблиці Б1 зі стовпчиком «сума» двійкового додавання одного розряду, а стовпчик AND — зі стовпчиком «перенесення». Що з цього випливає?

Блок В. Сигнали та рівні напруги · ≈ 10 хв

В1 Накресліть вертикальну шкалу від 0 до 5 В, позначте на ній три зони й підпишіть їхні межі. Потім укажіть, куди потрапляє кожна з напруг: 0,3 В; 0,9 В; 1,5 В; 4,2 В.

- B2** На вхід схеми послідовно надійшли напруги: $4,6 \cdot 4,1 \cdot 0,2 \cdot 0,5 \cdot 4,8 \cdot 0,1 \cdot 0,6 \cdot 4,3$ В. Запишіть вісім бітів, які прочитає схема. Яке це число в десятковій системі? А в шістнадцятковій?

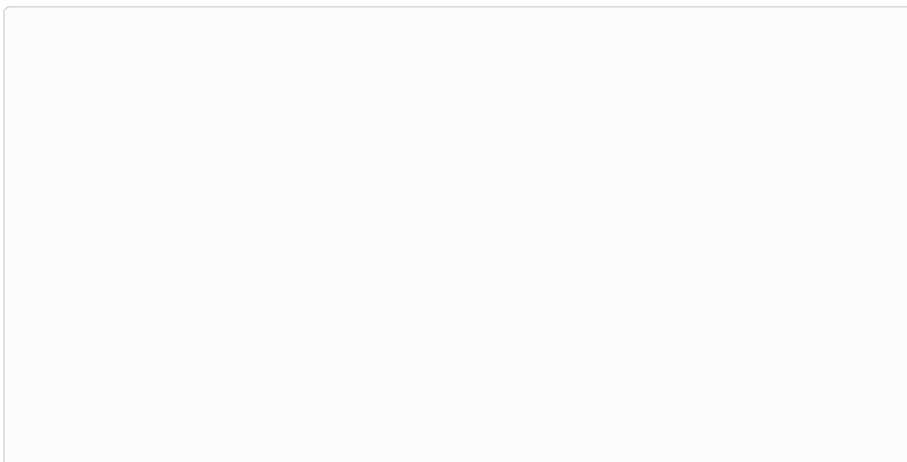
.....

.....

.....

.....

- B3** Намалуйте від руки два ряди по три графіки: аналоговий сигнал і цифровий сигнал — оригінал, після першого копіювання, після третього. Підпишіть, у якому ряду похибка накопичується і чому.



- B4** Уявіть, що замість двох рівнів вирішили зробити десять, рівномірно розподілених у діапазоні 0–5 В. Обчисліть крок між сусідніми рівнями й максимальний шум, який ще не сплутає рівні. Порівняйте це число з шириною забороненої зони у двійковій логіці.

.....

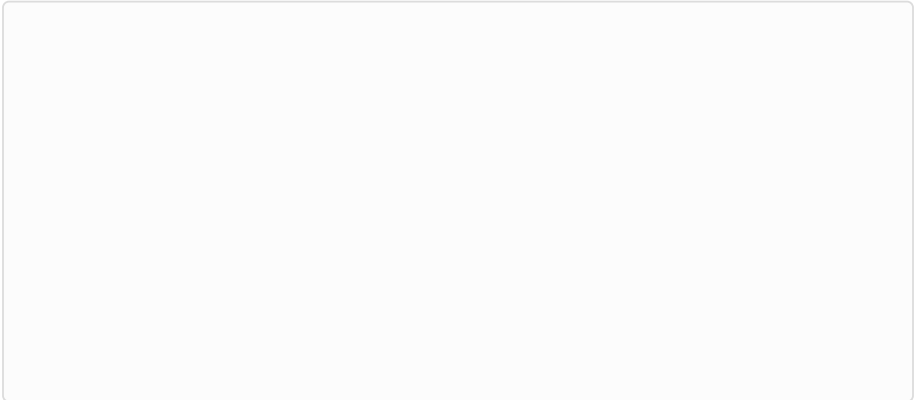
.....

.....

.....

Блок Г. Схеми та архітектура · ≈ 12 хв

- Г1** Намалуйте схему фон Неймана: чотири основні блоки та шину між ними. Усі підписи — українською.



- Г2** Розставте у правильному порядку знизу вгору вісім рівнів абстракції. Список навмисно перемішано:

.....

.....

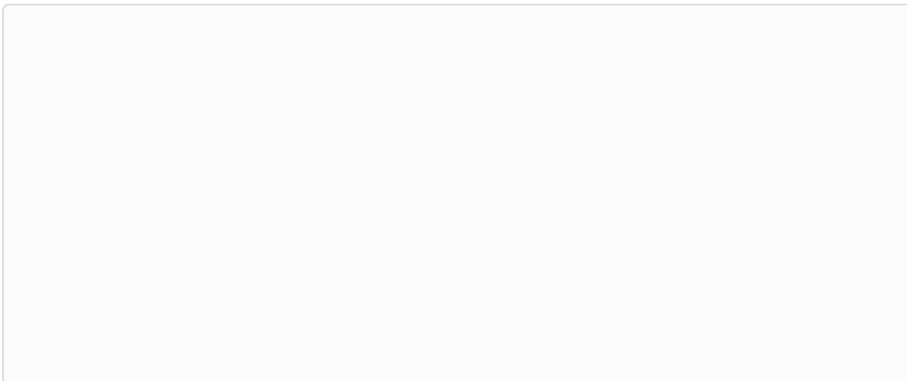
.....

.....

.....

- а) логічний елемент
- б) прикладна програма
- в) транзистор
- г) операційна система
- ґ) функціональний вузол
- д) архітектура системи команд
- е) фізика напівпровідників
- є) мікроархітектура

- Г3** Намалюйте цикл виконання команди — чотири кроки, замкнені в коло, з назвами українською та англійською.



- Г4** Для кожної схеми визначте: комбінаційна вона чи послідовнісна. Відповідь обґрунтуйте одним реченням.

.....

.....

.....

.....

.....

.....

.....

- а) суматор двох чотирирозрядних чисел
- б) лічильник відвідувачів магазину
- в) дешифратор
- г) кодовий замок на чотири цифри
- г) мультиплексор
- д) світлофор
- е) схема голосування із завдання Б4

- Г5** Фон Неймана чи гарвардська? Проставте навпроти кожної ознаки.

.....

.....

.....

.....

.....

.....

.....

- а) команди й дані лежать в одній пам'яті

- б) дві незалежні шини
- в) можна завантажити нову програму як звичайний файл
- г) типова для мікроконтролерів Arduino
- г) процесор може одночасно вибирати команду й дані

| Блок Д. Оцінки «на око» · ≈ 10 хв

Д1 Закон Мура. Intel 4004 (1971) містив 2300 транзисторів, кількість подвоюється кожні два роки. Скільки транзисторів мало б бути у процесорі 2021 року? Порівняйте свій результат із реальними «десятками мільярдів» і поясніть розбіжність.

Д2 Зображення 1920×1080 пікселів, 3 байти на піксель, без стиснення. Скільки це байтів? Приблизно скільки мебібайтів?

Д3 Виробник продає диск «1 ТБ», тобто 10^{12} байтів. Скільки це гібібайтів ($1 \text{ ГіБ} = 2^{30}$ байтів)? Чому система показує менше, ніж написано на коробці?

Д4 Процесор працює на частоті 3 ГГц. Одне звертання до оперативної пам'яті триває 100 нс. Скільки тактів процесор простоює, чекаючи на дані?

Д5 64-бітна адреса. Скільки різних комірок пам'яті можна нею заадресувати? Оцініть хоча б порядок числа.

.....

Відповіді та пояснення

Пояснення важливіші за самі відповіді: у кожному пункті вказано не лише «що», а й «чому саме так» і де студенти зазвичай помиляються.

Блок А

- A1** **19 = 10011₂**. Остачі при діленні: $19 \rightarrow 9$ (1), $9 \rightarrow 4$ (1), $4 \rightarrow 2$ (0), $2 \rightarrow 1$ (0), $1 \rightarrow 0$ (1).
Читаємо знизу вгору: 10011. Перевірка: $16+2+1 = 19$.
64 = 1000000₂. 64 — це рівно 2^6 , тому в записі одна одиниця й шість нулів.
100 = 1100100₂. Перевірка: $64+32+4 = 100$.
255 = 11111111₂. $255 = 2^8-1$, тому всі вісім розрядів — одиниці; це максимальне число, яке вміщується в один байт.
Головна помилка — записати остачі згори вниз. Правило самоперевірки: остання остача завжди 1 і завжди стає старшим розрядом, тож результат ніколи не починається з нуля. Разом варто навчитися впізнавати на око степені двійки (одна одиниця в записі) та числа «на один менше» від них (усі одиниці).
- A2** $1011_2 = 8+2+1 = 11$. $110110_2 = 32+16+4+2 = 54$. $10000000_2 = 128$. $11111111_2 = 255$.
Спосіб один: підписати над розрядами ваги 128, 64, 32, 16, 8, 4, 2, 1 і додати ті, під якими стоїть одиниця. Останні два числа варто просто запам'ятати — вони трапляються постійно.
- A3** $11010110_2 = 1101 \mid 0110 = \mathbf{D6}$. $01011010_2 = 0101 \mid 1010 = \mathbf{5A}$. $0x2F = 0010 \ 1111 = \mathbf{00101111_2}$. $0xC0 = 1100 \ 0000 = \mathbf{11000000_2}$.
Чому обчислювати нічого не треба: $16 = 2^4$, тому одна шістнадцяткова цифра відповідає рівно чотирьом бітам — завжди й без винятків. Розбивати на четвірки треба **справа наліво**; якщо ліворуч не вистачає розрядів, доповнюють нулями.
- A4** **1011 + 0110 = 10001₂ = 17**. По розрядах справа: $1+0=1$; $1+1=0$ і перенесення 1; $0+1+1=0$ і перенесення 1; $1+0+1=0$ і перенесення 1; залишок перенесення дає п'ятий розряд. Перевірка: $11+6 = 17$.
1111 + 0001 = 10000₂ = 16. Перевірка: $15+1 = 16$ — класичне «переповнення» чотирьох розрядів.
1100 + 1010 = 10110₂ = 22. Перевірка: $12+10 = 22$.
Правило одне-єдине: $1+1$ дає 0 у поточному розряді й перенесення в наступний — рівно так само, як $9+1$ у десятковій системі дає 0 і перенесення.
- A5** Шукаємо найменше n , для якого $2^n \geq N$.
а) 33 літери: $2^5 = 32$ — замало, $2^6 = 64 \rightarrow \mathbf{6 \text{ бітів}}$.
б) 7 днів: $2^2 = 4$ — замало, $2^3 = 8 \rightarrow \mathbf{3 \text{ біти}}$.
в) 365 днів: $2^8 = 256$ — замало, $2^9 = 512 \rightarrow \mathbf{9 \text{ бітів}}$.
г) 1000 студентів: $2^9 = 512$ — замало, $2^{10} = 1024 \rightarrow \mathbf{10 \text{ бітів}}$.
Типова помилка — ділити N на 2 і рахувати кроки. Правильна логіка інша: кожен доданий біт подвоює кількість станів, тому треба знайти найближчий степінь двійки **згори**. Зверніть увагу, наскільки «дешеві» біти: один зайвий біт — і замість 512 варіантів уже 1024.

- A6** **#FF8800** = FF, 88, 00 = **255 червоного, 136 зеленого, 0 синього**. FF = 15·16+15 = 255; 88 = 8·16+8 = 136.
rgb(0, 128, 255) = 00, 80, FF = **#0080FF**. Тут корисно запам'ятати, що 128 = 0x80: це рівно половина від 256, тому у шістнадцятковій вона виглядає «круглою», а в десятковій — ні.

Блок Б

- B1** Рядки в порядку 00, 01, 10, 11:
AND: **0, 0, 0, 1** — одиниця лише коли обидва входи одиниці.
OR: **0, 1, 1, 1** — нуль лише коли обидва нулі.
XOR: **0, 1, 1, 0** — одиниця, коли входи різні («або, але не обидва».)
NAND: **1, 1, 1, 0** — це AND, перевернутий на виході, тому його стовпчик у точності протилежний стовпчику AND.
Зручний спосіб не плутатися: спершу заповнити AND і OR, а NAND отримати простим перевертанням AND, не рахуючи заново.
- B2** Результат: **1, 1, 0, 1**.
A=0, B=0: (0 AND 0) OR (NOT 0) = 0 OR 1 = 1.
A=0, B=1: 0 OR 1 = 1.
A=1, B=0: (1 AND 0) OR (NOT 1) = 0 OR 0 = 0.
A=1, B=1: 1 OR 0 = 1.
Цей стовпчик збігається зі стовпчиком виразу (NOT A) OR B — тобто вираз можна спростити, викинувши з нього B у першій дужці. Це і є суть **мінімізації**, якою ми займатимемося окремо: та сама функція, менше елементів, дешевша й швидша схема.
- B3** 3 входи → 2³ = **8 рядків**; 4 входи → 2⁴ = **16**; 8 входів → 2⁸ = **256**. Формула: 2ⁿ.
Це той самий закон подвоєння, що й у завданні A5: кожен доданий вхід подвоює кількість комбінацій. Звідси й практичний висновок: таблицями істинності зручно працювати до 4–5 входів, далі вони стають некерованими, і потрібні інші інструменти.
- B4** У порядку 000 ... 111: **0, 0, 0, 1, 0, 1, 1, 1**. Одиниця стоїть у чотирьох рядках — 011, 101, 110 і 111, тобто там, де ввімкнено два або три перемикачі.
Ця схема називається **мажоритарним елементом** і має цілком реальне застосування: у системах із потрібним резервуванням три однакові блоки виконують ту саму роботу, а мажоритарний елемент бере той результат, який видали хоча б двоє. Відмова одного блока тоді не ламає систему. Так побудована електроніка літаків і космічних апаратів.
- B5** Стовпчик **XOR збігається зі стовпчиком суми**, а стовпчик **AND — зі стовпчиком перенесення**. Отже, два найпростіші логічні елементи разом уже вміють додавати один двійковий розряд. Ця пара має назву **півсуматор**, і саме з неї, наращуючи розрядність, будується вся арифметика процесора. Це хороша ілюстрація до головної думки курсу: складне збирається з дуже простого.

Блок В

- B1** Межі: від 0 до 0,8 В — логічний нуль; від 0,8 до 2,0 В — заборонена зона; від 2,0 до 5 В — логічна одиниця.

0,3 В — логічний 0. 0,9 В — заборонена зона. 1,5 В — заборонена зона. 4,2 В — логічна 1.

Пастка тут одна — 0,9 В: інтуїтивно це «майже нуль», але межу вже перейдено, і схема не має права трактувати таку напругу як визначений біт. Саме тому забороненою зоною сигнал має **проскакувати** якнайшвидше: поки він у ній, вихід схеми непередбачуваний.

- B2** $4,6 \rightarrow 1; 4,1 \rightarrow 1; 0,2 \rightarrow 0; 0,5 \rightarrow 0; 4,8 \rightarrow 1; 0,1 \rightarrow 0; 0,6 \rightarrow 0; 4,3 \rightarrow 1$. Отже, **11001001₂**.
У десятковій: $128+64+8+1 = 201$. У шістнадцятковій: $1100 | 1001 = 0xC9$
(перевірка: $12 \cdot 16 + 9 = 201$).

Головне, що має тут відкластися: схема **не вимірює** напругу, вона лише порівнює її з порогом. Напруги 4,1 і 4,8 дають однаковий біт, хоча відрізняються на 0,7 В; 0,2 і 0,6 — теж. Уся різниця між ними просто зникає, і разом із нею зникає накопичений шум.

- B3** В аналоговому ряду форма кривої з кожним копіюванням спотворюється дедалі сильніше, і **відновити оригінал уже неможливо**: доданий шум нічим не відрізнити від корисного сигналу. У цифровому ряду шум теж додається, але кожен логічний елемент порівнює сигнал із порогом і видає на виході **чистий, повний рівень 0 або 1**. Тому третя копія цифрового сигналу нічим не гірша за першу. Це явище називається **регенерацією сигналу**, і саме воно робить можливим побітово точне копіювання файлів — на відміну від перезапису аудіокасети, де кожна копія помітно гірша за попередню.

- B4** Десять рівнів у діапазоні 0–5 В дають дев'ять проміжків, тобто крок **$5/9 \approx 0,56$ В**. Сплутати сусідні рівні шум зможе вже тоді, коли перевищить половину кроку, тобто приблизно **0,28 В**.
У двійковій логіці заборонена зона має ширину 1,2 В (від 0,8 до 2,0) — запас більш ніж у чотири рази більший. Ось і вся відповідь на питання «чому не десять рівнів»: завадостійкість падає пропорційно кількості рівнів, а виграш у кількості інформації на один провідник зростає лише логарифмічно. Обмін не вигідний.

Блок Г

- G1** Чотири блоки: **арифметико-логічний пристрій (АЛП) і пристрій керування** (разом вони утворюють центральний процесор), **пам'ять і пристрої введення-виведення**. Усі з'єднані спільною **системною шиною**.
При перевірці звертайте увагу на дві речі. Перше: стрілка між процесором і пам'яттю має бути **двосторонньою** — дані ходять в обидва боки. Друге: пам'ять на схемі **одна**, і в ній лежать і команди, і дані — це і є принцип збереженої програми, головна ідея всієї моделі.

- G2** Знизу вгору: **фізика напівпровідників → транзистор → логічний елемент → функціональний вузол → мікроархітектура → архітектура системи команд → операційна система → прикладна програма**.

Найчастіша помилка — поміняти місцями мікроархітектуру й архітектуру системи

команд. Запам'ятати легко: система команд — це **контракт**, який видно програмісту ззовні, а мікроархітектура — це **реалізація** цього контракту всередині кристала, тому вона лежить нижче.

Г3 Вибірка (fetch) → декодування (decode) → виконання (execute) → запис результату (write back), і стрілка від четвертого кроку назад до першого. Важливо, що коло замкнене: процесор не «виконує програму» як щось цілісне — він мільярди разів на секунду повторює ці чотири кроки й нічого іншого не робить взагалі. Адресу наступної команди зберігає лічильник команд, який автоматично збільшується після кожної вибірки.

Г4 Комбінаційні: суматор, дешифратор, мультиплексор, схема голосування з Б4 — їхній вихід повністю визначається тим, що зараз на входах.

Послідовнісні: лічильник, кодовий замок, світлофор — їм необхідно пам'ятати передісторію.

Перевіряти зручно одним питанням: «чи може та сама комбінація входів дати різний результат?». Якщо може — схема послідовнісна. Лічильник при натисканні кнопки видає то 5, то 6 — залежно від того, що було раніше. Кодовий замок реагує на цифру 7 по-різному залежно від того, які цифри введено до неї. А от схема голосування при тих самих трьох перемикачах завжди дає той самий результат — вона комбінаційна.

Г5 Спільна пам'ять команд і даних — **фон Неймана**. Дві незалежні шини — **гарвардська**. Нову програму можна завантажити як звичайний файл — **фон Неймана** (саме тому на комп'ютері ви просто запускаєте завантажений застосунок). Типова для Arduino — **гарвардська**. Одночасна вибірка команди й даних — **гарвардська**.

Коротко: гарвардська виграє у швидкості, фон-нейманівська — у гнучкості. Тому універсальні комп'ютери роблять за фон Нейманом, а мікроконтролери з раз і назавжди прошитою програмою — за Гарвардом.

Блок Д

Д1 З 1971 до 2021 минуло 50 років, тобто **25 подвоєнь**. $2^{25} = 33\,554\,432$. Множимо: $2300 \times 33\,554\,432 \approx 7,7 \cdot 10^{10}$, тобто близько 77 мільярдів транзисторів. Реальні процесори 2021 року містять десятки мільярдів — порядок збігається, і це вже добрий результат для оцінки «на око». Але прогноз завищений, і причина відома: закон Мура помітно **сповільнився** приблизно з середини 2010-х, бо подальша мініатюризація впирається у фізичні межі. Подвоєння тепер відбувається рідше кожні 2,5–3 роки, ніж кожні 2.

Д2 $1920 \times 1080 = 2\,073\,600$ пікселів. Множимо на 3 байти: **6 220 800 байтів**. Ділимо на 1 048 576 (це 2^{20}) і отримуємо $\approx 5,93$ МіБ. Звідси видно, навіщо взагалі потрібне стиснення зображень: одна-єдина нестиснена світлина у Full HD важить майже шість мебібайтів, а година нестисненого відео на 30 кадрах за секунду — понад 600 гібібайтів.

Д3 $10^{12} \div 2^{30} = 10^{12} \div 1\,073\,741\,824 \approx 931,3$ ГіБ. Жодного обману тут немає — просто дві різні одиниці з майже однаковими назвами. Виробник накопичувача рахує чесними десятковими тисячами (1 ТБ = 10^{12} байтів), а операційна система показує двійкові гібібайти (2^{30} байтів), але

підписує їх звичним «ГБ». Різниця накопичується приблизно на 7 % на кожному кроці префікса, тому для терабайта вона й виходить помітною.

Д4 Період одного такту на 3 ГГц дорівнює $1 \div (3 \cdot 10^9) \text{ с} \approx \mathbf{0,33 \text{ нс}}$. Ділимо: $100 \text{ нс} \div 0,33 \text{ нс} \approx \mathbf{300 \text{ тактів}}$.

Тобто за час одного-єдиного звертання до оперативної пам'яті процесор міг би виконати близько трьохсот команд, але замість цього просто стоїть. Це і є **вузьке місце фон Неймана** в числах — і саме заради боротьби з цими трьомастами тактами існують кеш-пам'ять, конвеєр, передбачення переходів і багатоядерність, тобто добра половина нашого курсу.

Д5 $2^{64} \approx \mathbf{1,8 \cdot 10^{19}}$ — приблизно вісімнадцять із половиною квінтільйонів адрес. Щоб відчувати масштаб: якщо в кожній комірці лежить один байт, то 64-бітний адресний простір охоплює близько 16 ексабайтів. Це в мільйони разів більше за пам'ять будь-якого реального комп'ютера, тому переходити на 128 біт найближчим часом нікому не знадобиться — на відміну від переходу з 32 біт, де стеля у 4 гібібайти була досягнута дуже швидко.